Neighborhood Stability in Assignments on Graphs

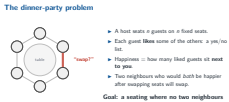
Haris Aziz, Grzegorz Lisowski, Mashbat Suzuki, Jeremy Vollen
arXiv:2407.00240, 2024

Presented by Ashik Shenoy (230224) and Achyuth H Warrier (230052)
CS666: Relational Structures in Games · IIT Kanpur · Instructor: Sunil Simon
11 November 2026

[Ashik · 0:30 · ends at 0:30]

Good morning. We are presenting "Neighborhood Stability in Assignments on Graphs" by Aziz, Lisowski, Suzuki and Vollen, posted on arXiv in 2024.

I will do the first half: the model, why stability can fail, and why letting people swap does not always work. Achyuth will do the second half: the algorithms that *do* work on cycles, paths, and a general family of graphs. Throughout, we will use one story (a dinner party) and one small four-guest example.



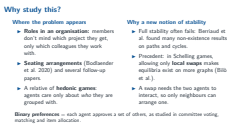
[Ashik · 0:45 · ends at 1:15]

Imagine you are hosting a dinner. Seats are fixed. Each guest has a simple yes/no opinion of every other guest: either they like them or they don't. A guest's happiness is just the number of liked guests sitting next to them.

Now the trouble: if two guests sitting next to each other would *both* be happier after swapping seats, they will lean over, agree, and swap. Then others may want to swap, and so on.

The host wants to avoid this: find a seating in which no two neighbours want to swap.

(The paper's introduction tells the same story with an organisation assigning members to roles; the dinner party is the version from the earlier literature, Bodlaender et al.)

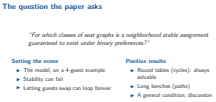


[Ashik · 1:00 · ends at 2:15]

Why should we care? The paper opens with an organisation example: roles are fixed in advance, members are happy with any project, but they care about which colleagues they work with. If two members would both rather swap roles, you get chaos. The same model was first studied for seating guests, by Bodlaender et al., and several recent papers followed. It's also closely related to hedonic games, where agents are partitioned into groups and only care about who is in their group.

Why a new stability notion? In hedonic games and in this seating model, stable outcomes often don't exist; Berriaud et al. found many non-existence results even on paths and cycles. A similar issue arose in Schelling segregation games, and Bilò et al. showed that allowing only local swaps makes equilibria exist on more graph classes. That's the inspiration here: only neighbours can swap, which is also realistic, since a swap needs the two people to talk.

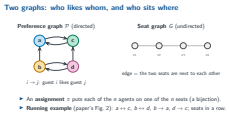
Binary preferences, approve or not, are a standard model in committee voting, matching and fair division.



[Ashik · 0:30 · ends at 2:45]

This is the question the paper asks, quoted directly. "Seat graph" is the shape of the table; "binary preferences" means the yes/no likes.

Plan: I'll set up the model on a small example, show that a good seating need not exist, and show that letting guests fix things themselves can cycle forever. Achyuth then shows the positive results: on a round table (a cycle) a good seating always exists and can be computed fast; similar for a bench (a path); and a general condition for other seat graphs.



[Ashik · 1:30 · ends at 4:15]

There are two separate graphs here, and keeping them apart is the key to the whole paper.

Left: the *preference graph*. It is directed. An arrow from i to j means i likes j. Liking need not be mutual.

Right: the *seat graph*. It is undirected, and has exactly n seats. An edge means two seats are next to each other. Here it is four seats in a row.

An assignment just puts each guest on one seat.

This is our running example, Figure 2 of the paper. a and c like each other, b and d like each other, and additionally b likes a and d likes c.

Please keep this picture in mind; we will reuse it many times.

Utility = number of liked neighbours

$u_i(\sigma) = |\{j \in N_\sigma(i) : i \to j\}|$ $\quad N_\sigma(i) =$ agents seated next to i



- a likes only c, who is far away: $u_a = 0$. d likes b next to it: $u_d = 1$.

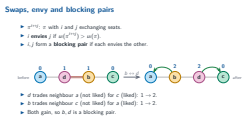
An additively separable hedonic game with 0/1 values, where your "group" is your seat neighbourhood.

[Ashik · 1:00 · ends at 5:15]

Formally, an agent's utility is the number of neighbours it likes. Nothing else counts.

Take the seating a, d, b, c in a row. a only likes c, who is at the other end, so a gets 0. d sits next to b, whom it likes: 1. b also likes d: 1. c likes only a: 0.

Course connection: we saw hedonic games where agents' value for a coalition is the sum of values for its members, i.e. additively separable. Here the values are 0 or 1. The difference is that the "group" you care about is not a coalition in a partition, but whoever ends up next to you on the seat graph. The paper itself points out this connection to hedonic games.



[Ashik · 1:30 · ends at 6:45]

Now the swap. Write $\pi^{i\leftrightarrow j}$ for the seating where i and j exchange seats. i envies j if i would be strictly happier in j's seat. A blocking pair is a pair where the envy goes both ways, so both would agree to swap.

Back to a, d, b, c. Look at b and d in the middle. If they swap, they are still next to each other, but d moves next to c, whom it likes, and b moves next to a, whom it likes. Both go from 1 to 2. So b, d is a blocking pair.

Notice the pattern: when two neighbours swap, each one just trades its *other* neighbour for the partner's other neighbour. Achyuth will use exactly this "trade one neighbour" view later.

These numbers are the paper's Example 1.



[Ashik · 1:00 · ends at 7:45]

This is the paper's central concept. A seating is *neighbourhood stable* (I'll say NS) if no blocking pair sits next to each other. The usual notion, simply "stable", forbids blocking pairs anywhere at the table.
Why is the restriction natural? A swap needs the two guests to agree, which means they must talk. And you can only talk to the people next to you, which are exactly the people who affect your happiness. The paper also argues that NS is natural when agents have limited information about the overall seating.
So our earlier seating a, d, b, c is not NS: b and d are neighbours and form a blocking pair.

How the notions relate



- Running example after the swap, a, b, d, c: **stable**, hence also NS.
- NS is the weakest of the three, so it is the most likely to exist.
- Even checking whether a stable seating exists is computationally hard.

[Ashik · 0:30 · ends at 8:15]

The notions are nested. Stable means no blocking pair anywhere. In between, the earlier paper by Berriaud et al. looked at forbidding blocking pairs within distance two. NS only forbids distance one. So every stable seating is NS, but not the other way round.

For our running example, the seating after the swap, a, b, d, c, is in fact fully stable, so certainly NS. That is stated in the paper's Example 1.

Since NS is the weakest requirement, it has the best chance of always existing. The paper also notes that deciding existence of a fully stable seating is computationally hard, inherited from the exchange-stable roommates problem, which is NP-complete.



[Ashik · 0:30 · ends at 8:45]

Why restrict to yes/no likes? Because anything richer fails almost immediately.

If an agent can value others at three levels, the paper points out, reusing a proof from Berriaud et al., that four agents on a cycle may have no NS seating. If dislikes are allowed, values plus or minus one, it fails even on a path of three seats. And two non-negative values reduce to binary. So binary preferences are the natural setting to look for positive results, and the rest of the talk is about binary preferences.



[Ashik · 1:30 · ends at 10:15]

Here is an example showing NS really is weaker than stability. Six agents, each likes exactly the next one around a directed 6-cycle. Seats: two separate tables of three, i.e. two triangles.

No stable seating exists: as the paper says, in every seating there are two agents at different tables who both have utility zero, and they would both gain by swapping, so they block. (Intuitively, a table of three can't contain everyone's single friend.)

But *every* seating is NS. Neighbours are at the same table, and in a triangle, swapping two people leaves everyone with exactly the same neighbours. Nobody gains.

This example is taken from Massand and Simon, IJCAI 2019, *Graphical one-sided markets*. Sunil Simon is our instructor.

We also brute-forced it: all 720 seatings are NS, none is stable.



[Ashik · 1:00 · ends at 11:15]

Now the bad news. Theorem 1: for every even $n = 2t$ with t odd and at least 3, there's an instance with no NS seating at all.

The construction: seats form the complete bipartite graph $K_{t,t}$, two sides, every seat on one side next to every seat on the other side. Like two rows facing each other where everyone talks to everyone across. Agents come in two rows, top s_{1j} and bottom s_{2j}. Each agent likes exactly two people: its twin in the other row (the vertical 2-cycles), and its successor in its own row (the two directed t-cycles). Here $t = 3$, which is the paper's Figure 1 with six agents.

We verified the six-agent case by brute force: none of the 720 seatings is NS.



[Ashik · 1:15 · ends at 12:30]

Here is the trick that makes the proof manageable. In a complete bipartite seat graph, your neighbours are *all* the people on the opposite side. So the only thing that matters is which side each agent is on.

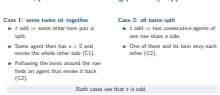
Each agent likes exactly two others, so utility is 0, 1 or 2: the number of its two favourites who are across.

If i swaps with some j across, i moves to the other side; its new neighbours are its old side-mates plus j. That gives two simple envy conditions.

C1: if both of i's favourites are on i's own side, i has 0 and would get 2 by moving across: i envies everyone across.

C2: if one favourite is on i's side and the other favourite is j across, then swapping with j gives 2 instead of 1.

The proof shows that in every split, some pair envy each other this way. Since they're on opposite sides, they're adjacent.



[Ashik · 1:45 · ends at 14:15]

Two cases, depending on whether some twins sit on the same side. Case 2 is the cleaner one; start there. Suppose every twin pair is split. Then each side holds exactly one of each pair, so at least $\lceil t/2 \rceil$ agents on one side come from the same row, say the top. They lie on a directed t-cycle; an odd cycle has no independent set larger than $\lfloor t/2 \rfloor$, so two consecutive top agents $s_{1j}, s_{1,j+1}$ are on the same side. Then s_{1j} likes $s_{1,j+1}$ on its side and likes its twin across: C2, it envies its twin. Symmetrically the twin s_{2j} likes its successor (which is across from $s_{1,j+1}$, on its own side) and likes s_{1j}: C2. Blocking pair.

Case 1: some twins are together. Parity forces some split twin, so we can find j where twins j are together and twins $j+1$ are split. That s_{1j} has both favourites on its side: utility 0, envies everyone across. To avoid an immediate blocking pair, nobody across may have 0. That forces a pattern: every together-pair on the right is followed by a together-pair on the left. Counting, the pair just before j must be together on the right, and its top agent envies s_{1j} by C2. Blocking pair.

Main message: oddness is used in both cases.



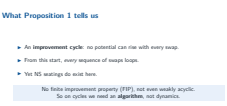
[Ashik · 1:45 · ends at 16:00]

A natural idea: start anywhere and let neighbours who form a blocking pair swap, until nobody wants to. The paper calls this swap dynamics. Proposition 1 says this can fail even on a round table with four seats.

Different preferences here (the paper's Fig. 6, left): c likes a; a likes b and d; b likes d; d likes c.

Start with a, b, c, d around the square. b and c have 0 each; if they swap, each gets 1. Swap. Now a and d block: a goes from 1 to 2, d from 0 to 1. Swap. Then b, c again, then a, d again, and we're back where we started.

Crucially, the paper checks that the highlighted pair is the *unique* blocking pair at every step. So it's not bad luck in choosing which pair swaps; there is no other choice. (Our script confirms the loop and the uniqueness.)



[Ashik · 1:00 · ends at 17:00]

Let me connect this to what we did in class.

A loop of strictly improving moves is an improvement cycle. In potential games, the finite improvement property, FIP, is equivalent to having a generalised ordinal potential. So, by analogy, there can be no potential function for swap dynamics on cycles. In fact it's worse: because each blocking pair is unique, from this start *no* sequence of swaps ever terminates. So it's not even weakly acyclic, unlike the colouring games we saw.

But the instance does have NS seatings (the paper says so; we counted 8 of the 24). So for cycles we need a constructive argument, not dynamics.

Contrast, for when swaps do work: the paper remarks that symmetric preferences guarantee NS. Our own quick argument: when likes are mutual, a blocking swap increases the number of adjacent mutually-liking pairs by exactly the two swappers' total gain, so it's a potential.

Handover: Achyuth will now show how to construct NS seatings.

Map of results

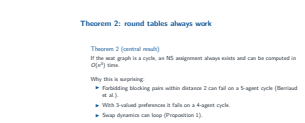
Seat graph	Guarantee	Algorithm
$K_{t,t}$, $t \geq 3$ odd	NS may not exist (Thm 1)	–
Cycle	NS always exists (Thm 2)	$O(n^3)$
Path	no blocking pair within distance 2 (Thm 3)	$O(n^3)$
Enough leaves	NS exists (Thm 4)	poly. if $\mathcal{P}$ is a DAG

DFVS – directed feedback vertex set (explained with Theorem 4); DAG – directed acyclic graph.
$O(n^3)$, $O(n^5)$: running time grows at most like the square / cube of the number of agents.

[Achyuth · 1:00 · ends at 18:00]

Thanks, Ashik. Here's the map of what the paper proves.
Ashik showed the negative result on complete bipartite seat graphs. The positive results are:

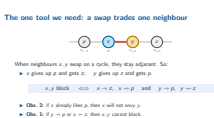
- On a cycle, a round table, an NS seating always exists, computed in cubic time. This is the central result and I'll spend most time on it.
- On a path, a bench, something even stronger: no blocking pair within distance two, also cubic time.
- On any seat graph, if it has enough leaves, measured against how far the preference graph is from acyclic, NS exists; and when the preferences have no cycles at all, it's computable in polynomial time.



[Achyuth · 0:30 · ends at 18:30]

Theorem 2: on a cycle, an NS seating always exists, and we can find one in $O(n^3)$ time.

The paper calls this surprising, and here's why. Berriaud et al. showed that if you also forbid blocking pairs at distance two, it can fail with just five agents on a cycle. With three-valued preferences it fails on four agents. And as Ashik showed, the swap dynamics can loop. Yet with binary preferences and only adjacent swaps, it always works.



[Achyuth · 1:30 · ends at 20:00]

Everything on cycles rests on one simple observation, which Ashik hinted at.

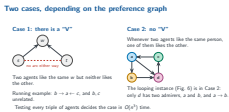
Take two neighbours x and y on the cycle, with p on x's other side and z on y's other side. If x and y swap, they're still next to each other.

The only change for x is that it loses p and gains z. For y, it loses z and gains p.

So x envies y exactly when x likes z but not p. And y envies x exactly when y likes p but not z. A blocking pair needs all four conditions.

The paper states this as two observations. Observation 2: if you already like the neighbour you'd lose, you won't envy. Observation 1: if y doesn't like p, or x doesn't like z, no blocking pair.

Check with Ashik's example: d was x, b was y, $p = a$, $z = c$. d likes c not a; b likes a not c. All four hold: blocking.



[Achyuth · 1:00 · ends at 21:00]

Algorithm 1 splits into two cases based only on the preference graph.
Case 1: there's a "V". Two agents s and t both like the same w, but there's no arrow between s and t in either direction. Our running example has one: b and c both like a, and b, c are unrelated.
Case 2: no V. Whenever two people like the same person, one of them likes the other. Ashik's looping example falls here: only d has two admirers, a and b, and a likes b.
Checking which case we're in just means looking at all triples: cubic time.



[Achyuth · 1:30 · ends at 22:30]

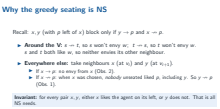
Case 1 is solved by a simple greedy rule.

Put the V on the first three seats: s, then w, then t. Then go round the table: for each next seat, pick an unseated agent who likes the agent in the previous seat, if possible; otherwise anyone.

On our running example: $s = b$, $w = a$, $t = c$. For seat 4 we need an unseated agent who likes c. Only d is left, and d does like c. So the table is b, a, c, d.

Utilities: b sits between a and d, likes both: 2. a is next to c: 1, and a only likes c, so that's its max. Same for c. d is next to b and c: 2. Everyone is at their maximum, so clearly NS. We verified this in code too.

Note on this picture: green arrows are likes between neighbours.



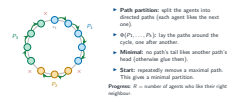
[Achyuth · 1:15 · ends at 23:45]

Why does this work? Use the swap rule: x and y block only if y likes p while x doesn't.

Around the V it's by design: s doesn't like t, so swapping with w (gaining t) is useless for s. Likewise t doesn't like s. And s, t both like w, so they won't give w up by swapping with their other neighbours.

Everywhere else, the greedy invariant does the job. When we chose x for seat v_i, we picked someone who likes p if anyone could. If x does not like p, then nobody unseated liked p at that time, and y was unseated then. So y doesn't like p either, and y has no reason to swap. Either way, no blocking pair.

The V is needed to start the cycle safely: the first and last seats wrap around.



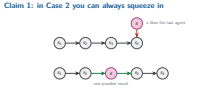
[Achyuth · 1:30 · ends at 25:30]

Case 2 is the interesting one, and it's the paper's new technique: path partitions.

Split all agents into directed paths in the preference graph: each agent on a path likes the next one. Then lay these paths around the table one after another; the paper calls this seating Φ. This picture is like the paper's Figure 4. Inside a path, everyone likes their right-hand neighbour (green arrows). The red crosses are the boundaries, where a path's tail sits next to the next path's head.

A partition is minimal if no tail likes another path's head. If one did, we'd just glue the two paths together. To start, take a maximal path, remove it, repeat; that is automatically minimal.

Keep in mind the progress measure: the number of agents who like their right-hand neighbour. Fewer paths means more of these. The algorithm will strictly increase this number.

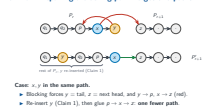


[Achyuth · 0:45 · ends at 26:00]

One helper fact for Case 2. Suppose an outsider s likes the last agent of a path. Then s can be inserted somewhere into the path while keeping it a path.

Why? s and s_{r-1} both like s_r. In Case 2 there's no V, so one of them likes the other. If s_{r-1} likes s, insert s right there: $s_{r-1} \to s \to s_r$. Otherwise s likes s_{r-1}, and we repeat the argument one step back. Eventually we either insert or reach the front, and s likes s_1 so we prepend. Formally it's an induction on the path length.

Note this is exactly where Case 2's "no V" property gets used.



[Achyuth · 2:00 · ends at 28:00]

This is the heart of the cycle proof. Suppose the seating from our path partition has a blocking pair x, y. Lemma 1 says we can build a new minimal partition with fewer paths.

Type I: x and y are consecutive in the same path. By the swap rule, y must not like z, its right neighbour. Inside a path everyone likes the next one, so y must be the *tail* of its path and z the head of the next path. The swap rule also says y likes p and x likes z: those are the two red arrows.

Repair, as in the paper's Figure 5: remove x and y from the end of P_r. The remaining path ends at p, and y likes p, so by Claim 1 we can re-insert y somewhere. The path still ends at p; p likes x; and x likes z. So we can glue everything onto P_{r+1}. Two paths became one.

If P_r has only two agents, it's similar: y goes into the previous path and x goes in front of the next one.

Then make it minimal again by gluing whatever can be glued. Fewer paths means more agents liking their right-hand neighbour.



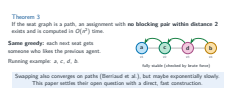
[Achyuth · 1:30 · ends at 29:30]

The other type of blocking pair straddles a boundary: x is the tail of one path, y the head of the next. The swap rule needs y not to like z. But if y's path had a second agent, y would like it. So y is alone, and z is the head of the path after that. And x must like z: a tail liking another head, contradicting minimality. So Type II can't happen.

Termination: the number of agents who like their right neighbour is at most n, and each repair raises it. So at most n repairs, each quadratic, cubic overall.

On Ashik's looping instance, if the algorithm's maximal path is $c \to a \to b \to d$, it covers everyone, d likes c, and wrapping it round the table is already NS.

Course link: this is a progress-measure argument like the energy in Hopfield networks, but the measure has at most $n+1$ values, so no exponential PLS-style search. Note the moves here are the algorithm's re-arrangements, not guests' swaps.



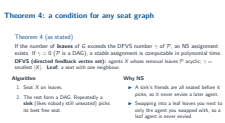
[Achyuth · 1:30 · ends at 31:00]

For a bench, a path seat graph, the paper gets something stronger than NS: no blocking pair within distance two. Running time: each of the n seats needs one scan of the unseated agents, so $O(n^2)$. (If asked: the paper's theorem states $O(n^3)$, which is a looser bound; the greedy itself only needs $O(n^2)$.)

The algorithm is the same greedy: start anywhere, and each next seat goes to someone who likes the previous person, if possible. Running example: start with a; c likes a; d likes c; b likes d. Seating a, c, d, b. We checked this one is even fully stable.

Why it works, roughly. Look at agent x and the two agents to its right. If x likes its left neighbour, the only way to gain is to reach utility 2, which would need liking its right neighbour, and then it already has 2. If x doesn't like its left neighbour, the greedy rule says nobody chosen after x liked that neighbour either. So the agents one or two seats to the right gain nothing by taking x's seat. The first seat is never envied since it's an endpoint.

Course connection: Berriaud et al. had shown, via a potential function, that swap dynamics converge on paths. But they gave evidence it can take exponentially long, which is the PLS phenomenon we saw. They



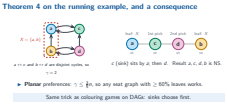
[Achyuth · 1:15 · ends at 32:15]

Finally, a result for general seat graphs. It compares two numbers. On the seat side: the number of leaves, seats with only one neighbour, like the end of a bench. On the preference side: the DFVS number, the smallest number of agents you must remove so the preference graph has no directed cycle.

Theorem: if there are enough leaves, an NS seating exists. And if the preference graph has no cycles at all, you even get a fully stable seating in polynomial time.

The algorithm: put the troublesome agents X on leaves. The rest form a DAG. Take a sink, an agent who likes nobody still unseated, and let it choose its favourite free seat. Repeat.

Why does it work? When a sink chooses, everyone it likes is already seated, so later moves can't make another seat better for it: it never envies a later agent. So any blocking pair involves an X agent on a leaf. But if you swap into a leaf, your only neighbour becomes the agent you swapped with, whom you already had. So no gain.



[Achyuth · 0:45 · ends at 33:00]

On our running example: the two mutual pairs are disjoint cycles, so we must remove two agents, say a and b; $\gamma = 2$. A path of four seats has two leaves. Put a and b on the ends. Among c and d, d likes c, so c is the sink and picks first: it likes a, so it sits next to a. Then d takes the last seat, next to both c and b. We get a, c, d, b, the same as the greedy path seating.

Consequence from the paper: planar preference graphs have DFVS at most three-fifths of n, so any seat graph that's at least 60% leaves works, for example full m-ary trees with m at least 3.

Course link: this is the same trick as for colouring games on DAGs: let the sinks choose first, in reverse topological order.

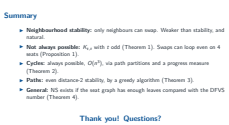
[Achyuth · 1:00 · ends at 34:00]

To close, the open directions the paper itself lists in its conclusion.

First, other seat graphs: does NS always exist on grids or trees? And more results like Theorem 4 would help describe which instances always have an NS seating.

Second, efficiency: how much total happiness is lost by insisting on NS, the price of neighbourhood stability, or how to find the happiest seating among the NS ones.

Third, richer preferences. On cycles, NS can already fail with cardinal values, as we saw. On paths, the question is open.



[Achyuth · 0:30 · ends at 34:30]

To summarise. Neighbourhood stability only lets neighbours swap; it's weaker than stability and natural for seating. It can fail, on complete bipartite seat graphs with odd t, and swap dynamics can loop even on four seats. But on round tables an NS seating always exists and is found in cubic time using path partitions. On benches we even get distance-two stability by a greedy algorithm. And for general seat graphs, having enough leaves compared to the DFVS number suffices. Thank you, we're happy to take questions.

(Backup if asked: our brute-force scripts checked every 4-agent instance on the 4-cycle and 6,000 random 5–6 agent instances; no counterexample to Theorem 2.)